

C Programming

[INDEX](#)[◀ Previous](#)[Next ▶](#)

POINTERS

Pointers enable us to effectively represent complex data structures, to change values as arguments to functions, to work with memory which has been dynamically allocated, and to more concisely and efficiently deal with arrays. A pointer provides an indirect means of accessing the value of a particular data item. Lets see how pointers actually work with a simple example,

```
int count = 10, *int_pointer;
```

declares an integer *count* with a value of 10, and also an integer pointer called *int_pointer*. Note that the prefix *** defines the variable to be of type pointer. To set up an indirect reference between *int_pointer* and *count*, the *&* prefix is used, ie,

```
int_pointer = &count
```

This assigns the memory address of *count* to *int_pointer*, not the actual value of *count* stored at that address.

POINTERS CONTAIN MEMORY ADDRESSES, NOT VALUES!

To reference the value of *count* using *int_pointer*, the *** is used in an assignment, eg,

```
x = *int_pointer;
```

Since *int_pointer* is set to the memory address of *count*, this operation has the effect of assigning the contents of the memory address pointed to by *int_pointer* to the variable *x*, so that after the operation variable *x* has a value of 10.

```
#include <stdio.h>

main()
{
    int count = 10, x, *int_pointer;

    /* this assigns the memory address of count to int_pointer */
    int_pointer = &count;

    /* assigns the value stored at the address specified by int_point
    x = *int_pointer;

    printf("count = %d, x = %d\n", count, x);
}
```

This however, does not illustrate a good use for pointers.

The following program illustrates another way to use pointers, this time with characters,

```
#include <stdio.h>

main()
{
    char c = 'Q';
    char *char_pointer = &c;

    printf("%c %c\n", c, *char_pointer);

    c = 'Z';
    printf("%c %c\n", c, *char_pointer);
    *char_pointer = 'Y';
    /* assigns Y as the contents of the memory address specified by c */

    printf("%c %c\n", c, *char_pointer);
}
```

©Copyright B Brown. 1984-1999. All rights reserved.

